

Python 编程与人工智能

进阶 Python 基础

Chengwei QIN

助理教授 @ AI Thrust

异常

- 异常层次结构、异常处理

函数

- 基础、形参与实参、匿名 (Lambda) 函数



異常

错误信息

上半部分显示错误上下文——堆栈回溯，指出错误发生的位置
最后一行显示错误类型以及对出错原因的简短说明

```
Traceback (most recent call last):  
  File "/Users/sunying/Desktop/Python class/week1/class_27/  
project_1/mymodule.py", line 8, in <module>  
    print(cou) 该语句  
NameError: name 'cou' is not defined
```

确切位置

错误

两类主要错误：

- 语法错误：代码不是合法 **Python** 时发生（也称为解析错误）。

```
>>> while True print('Hello world')
      File "<stdin>", line 1
        while True print('Hello world')
                        ^
SyntaxError: invalid syntax
```

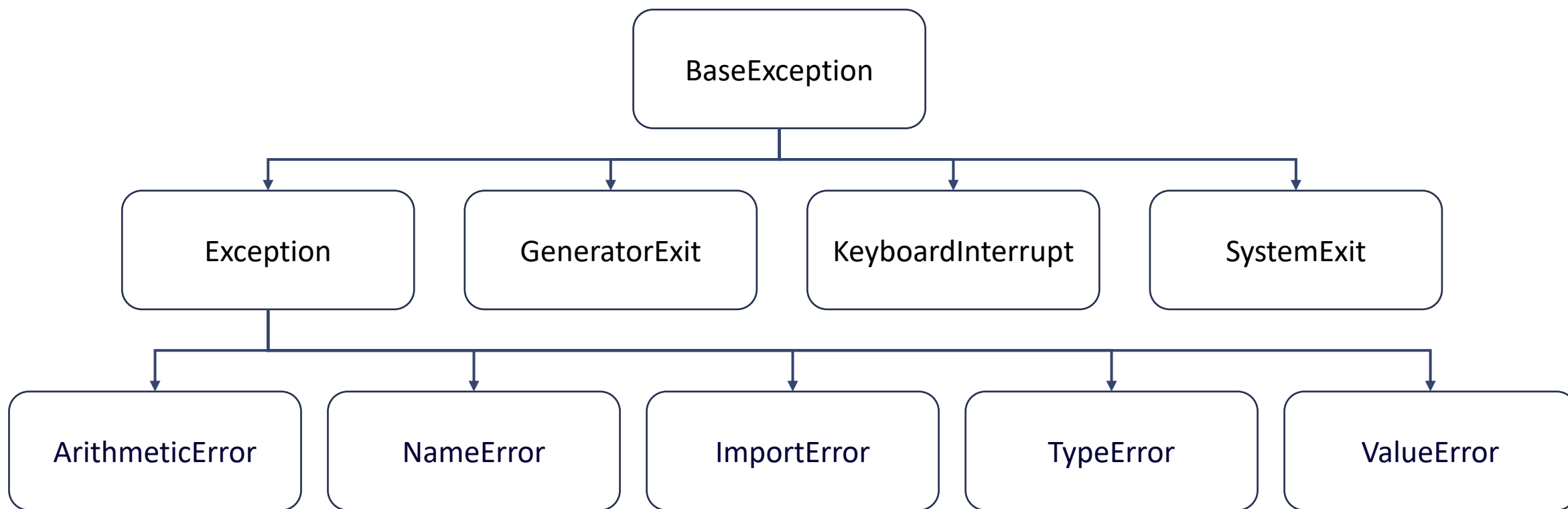
- 运行时错误（异常）：代码语法正确，但执行时出错
 - 可能是致命的，取决于如何处理它们
 - 常见原因包括无效输入、函数使用不当，或网络故障等外部问题

Python 在执行过程中可能抛出多种不同类型的错误

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

这些信息指出所发生 Exception 的具体类型

- 在 Python 中，错误以异常的形式表示
- 因为 Python 中一切皆对象，异常也是对象
- 异常通过继承形成层次结构



- ArithmeticError
 - FloatingPointError
 - OverflowError
 - ZeroDivisionError
- AssertionError
- AttributeError
- BufferError
- EOFError
- ImportError
 - ModuleNotFoundError
- LookupError
 - IndexError
 - KeyError
- MemoryError
- NameError
 - UnboundLocalError
- OSError
 - BlockingIOError
 - ChildProcessError
 - ConnectionError
 - BrokenPipeError
 - ConnectionAbortedError
 - ConnectionRefusedError
 - ConnectionResetError
 - FileExistsError
 - FileNotFoundError
 - InterruptedError
 - IsADirectoryError
 - NotADirectoryError
 - PermissionError
 - ProcessLookupError
 - TimeoutError
- ReferenceError
- RuntimeError
 - NotImplementedError
 - RecursionError
- StopIteration
- StopAsyncIteration
- SyntaxError
 - IndentationError
 - TabError
- SystemError
- TypeError
- ValueError
 - UnicodeError
 - UnicodeDecodeError
 - UnicodeEncodeError
 - UnicodeTranslateError
- Warning

异常处理

在 Python 中，出现意外情况时会抛出异常

- 默认情况下，未处理的异常会终止程序
- Python 提供显式处理异常的机制，使程序可以恢复或采取替代措施，而不是直接崩溃
- 异常处理中使用的关键字：
 - try, except, else, finally, raise, assert

```
try_stmt ::= try1_stmt | try2_stmt | try3_stmt
try1_stmt ::= "try" ":" suite
              ("except" [expression ["as" identifier]] ":" suite)+
              ["else" ":" suite]
              ["finally" ":" suite]
try2_stmt ::= "try" ":" suite
              ("except" "*" expression ["as" identifier] ":" suite)+
              ["else" ":" suite]
              ["finally" ":" suite]
try3_stmt ::= "try" ":" suite
              "finally" ":" suite
```

try-except 语句

处理代码特定部分中可能出现的异常

- 若 `try` 块中发生错误，则执行对应的 `except` 块
- 这可以防止程序意外崩溃

语法

`try` 块中发生的异常由对应的 `except` 块捕获并处理

```
01. try:
02.     statement(s)
03. except:
04.     error-handling-statement(s)
```

`except` 块仅在捕获到异常时运行；若未发生异常则跳过

try-except 语句

示例：一个简单的除法函数

```
def divide(n, m):  
    print('Let\'s start the Division!')  
    return n / m
```

✓ 3.9s

Python

4 除以 1

```
print(divide(4, 1))
```

✓ 0.0s

```
Let's start the Division!  
4.0
```

用 4 除以 0，出错！

```
print(divide(4, 0))
```

⊗ 0.0s

```
Let's start the Division!
```

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
/var/folders/1p/9bcd1_s73b6cmjvy82yc_vr0000gn/T/ipykernel_99575/1158075214.py in <module>  
----> 1 print(divide(4, 0))  
  
/var/folders/1p/9bcd1_s73b6cmjvy82yc_vr0000gn/T/ipykernel_99575/3250101075.py in divide(n, m)  
      1 def divide(n, m):  
      2     print('Let\'s start the Division!')  
----> 3     return n / m  
  
ZeroDivisionError: division by zero
```

try-except 语句

示例

- 现在用 try-except 语句添加异常处理
- return a large number if divided by zero

```
def safe_divide(n, m):  
    try:  
        print('Let\'s start the Division!')  
        print('result:', n / m)  
    except:  
        print('result:', 1e100)
```

正常运行

```
safe_divide(4, 1)
```

✓ 0.0s

```
Let's start the Division!  
result: 4.0
```

错误处理语句被执行了

```
safe_divide(4, 0)
```

✓ 0.0s

```
Let's start the Division!  
result: 1e+100
```

```
safe_divide(4, '2')
```

✓ 0.0s

```
Let's start the Division!  
result: 1e+100
```

如果发生其他错误呢？

捕获特定异常

不同情况下可能发生不同的错误

- 示例：用数字除以字符串时输出 1e100

```
def safe_divide(a, b):  
    try:  
        return a / b  
    except:  
        return 1E100  
  
print(safe_divide(1, '2'))
```

✓ 0.0s

1e+100

捕获特定异常

只处理特定类型的异常

- 整数与字符串相除会引发 `TypeError`
- 除以零会引发 `ZeroDivisionError`

语法

```
01. try:
02.     expression
03. except ErrorType:
04.     expression
```

捕获特定异常

示例：只处理 ZeroDivisionError

```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except ZeroDivisionError:  
05.         return 1E100
```

```
print(safe_divide(2, 0))
```

✓ 0.1s

Python

1e+100

```
print(safe_divide(1, 2))
```

✓ 0.1s

Python

0.5

TypeError 未被处理

```
print(safe_divide(1, '2'))
```

⊗ 0.2s

Python

TypeError

Traceback

ck (most recent call last)

Cell In[11], line 1

----> 1 print(safe_divide(1, '2'))

Cell In[8], line 3, in safe_divide(a, b)

1 def safe_divide(a, b):

2 try:

----> 3 return a / b

4 except ZeroDivisionError:

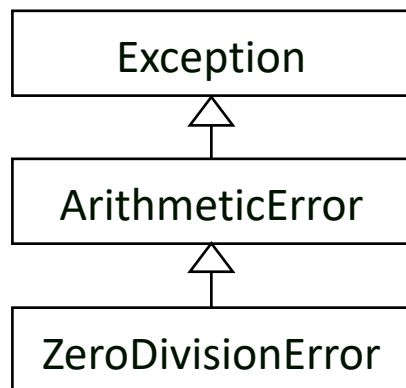
5 return 1E100

TypeError: unsupported operand type(s) for /: 'int' and 'str'

捕获特定异常

继承关系很重要

- 示例：ZeroDivisionError 同时也是 ArithmeticError 和 Exception



```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except ArithmeticError:  
05.         return 1E100
```

```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except ZeroDivisionError:  
05.         return 1E100
```

```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except Exception:  
05.         return 1E100
```

使用 `as` 关键字访问被引发的异常对象

语法

```
01. try:
02.     expression
03. except ErrorType as e:
04.     expression
```

可用变量保存异常对象。它可以是任意合法标识符

使用该对象来检查或处理错误

访问异常对象

使用 `as` 关键字访问被引发的异常对象

示例：捕获所有异常并检查它们的类型

```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except Exception as e:  
05.         return e  大类
```

异常对象

- 获取类型: `type()`
- 获取描述: `str()`

```
def safe_divide(a, b):  
    try:  
        return a / b  
    except Exception as e:  
        return e  
  
a = safe_divide(1, 0)  
b = safe_divide(1, '2')
```

```
print(a, type(a))  
print(b, type(b))  
a
```

✓ 0.0s

```
division by zero <class 'ZeroDivisionError'>  
unsupported operand type(s) for /: 'int' and 'str' <class 'TypeError'>  
  
ZeroDivisionError('division by zero')
```

处理特定异常

用不同操作处理不同类型的异常

- 允许使用多个 `except` 块

语法

```
01. try:
02.     expression
03. except ErrorType as e:
04.     expression
05. except ErrorType as e:
06.     expression
```

执行时机：捕获所需类型

执行条件：

- 先前期望的类型未被捕获
- 当前期望的类型被捕获
- 注意：继承关系很重要

处理特定异常

示例

分别以不同方式处理 `TypeError` 和 `ZeroDivisionError`

```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except ZeroDivisionError:  
05.         return 1E100  
06.     except TypeError:  
07.         print('Not valid input')  
08.         return 0
```

```
print(safe_divide(2, 0))
```

✓ 0.1s

1e+100

```
print(safe_divide(1, '2'))
```

✓ 0.1s

Not valid input

0

else 语句

- 仅当 try 块中未引发异常时才执行

语法

```
01. try:
02.     expression
03. except ErrorType [as e]:
04.     expression
05. except ErrorType [as e]:
06.     expression
07. else:
08.     expression
```

执行条件：
未发生异常

else 语句

- 示例

示例

```
01. def safe_divide(a, b):  
02.     try:  
03.         c = a / b  
04.     except ZeroDivisionError:  
05.         return 1E100  
06.     except TypeError:  
07.         print('Not valid input')  
08.         return 0  
09.     else:  
10.         print('No exception')  
11.         return c
```

```
print(safe_divide(1, 2))
```

✓ 0.2s

Python

No exception

0.5

```
print(safe_divide(2, 0))
```

✓ 0.1s

Python

1e+100

```
print(safe_divide(1, '2'))
```

✓ 0.1s

Python

Not valid input

0

finally 语句

- finally 块在 try-except 结构之后总会执行

语法

```
01. try:
02.     expression
03. except ErrorType [as e]:
04.     expression
05. except ErrorType [as e]:
06.     expression
07. else:
08.     expression
09. finally:
10.     expression
```

仍会执行

- 无论是否捕获异常

finally 语句

- finally 块在 try-except 结构之后总会执行

语法

```
01. def safe_divide(a, b):  
02.     try:  
03.         c = a / b  
04.     except ZeroDivisionError:  
05.         c = 1E100  
06.     except TypeError:  
07.         print('Not valid input')  
08.         c = 0  
09.     else:  
10.         print('No exception')  
11.     finally:  
12.         print('Ended')  
13.         return c
```

```
print(safe_divide(1, 2))  
✓ 0.2s Python  
No exception  
Ended  
0.5
```

```
print(safe_divide(2, 0))  
✓ 0.1s Python  
Ended  
1e+100
```

```
print(safe_divide(1, '2'))  
✓ 0.1s Python  
Not valid input  
Ended  
0
```

```
try:
    print("try something here")
except:
    print("this happens only if it fails")
else:
    print("this happens only if it succeeds")
finally:
    print("this happens no matter what")
```

✓ 0.2s

```
try something here
this happens only if it succeeds
this happens no matter what
```

Try: 捕获异常

Except: 处理异常

Else: 未发生异常时执行

Finally: 无论如何都会执行

使用 raise 语句显式触发指定异常

语法

```
raise error_type(error_message)
```

异常类型

异常对象

消息（字符串）

示例：抛出名称错误

```
raise NameError('My Error Messgae')
```

⊗ 0.2s Python

```
NameError                                Traceback  
  (most recent call last)  
  Cell In[33], line 1  
----> 1 raise NameError('My Error Messgae')
```

NameError: My Error Messgae

■ 抛出异常

示例：生成斐波那契数列

```
def fibonacci(N):  
    L = []  
    a, b = 0, 1  
    while len(L) < N:  
        a, b = b, a + b  
        L.append(a)  
    return L  
print(fibonacci(10))  
print(fibonacci(-10))
```

✓ 0.2s

[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
[]

- 输入值不能为负数
- 不抛出任何错误
- 空列表之后可能引发问题

抛出异常

抛出 ValueError，告知程序员不支持负数 N

```
def fibonacci(N):  
    if N < 0:  
        raise ValueError("N must be non-negative")  
    L = []  
    a, b = 0, 1  
    while len(L) < N:  
        a, b = b, a + b  
        L.append(a)  
    return L  
print(fibonacci(10))  
print(fibonacci(-10))
```

⊗ 0.2s

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[7], line 11  
      9     return L  
     10 print(fibonacci(10))  
----> 11 print(fibonacci(-10))  
  
Cell In[7], line 3, in fibonacci(N)  
      1 def fibonacci(N):  
      2     if N < 0:  
----> 3         raise ValueError("N must be non-negative")  
      4     L = []  
      5     a, b = 0, 1  
  
ValueError: N must be non-negative
```

■ 抛出异常

一种简单做法: raise 一个 AssertionError

语法 返回布尔值的表达式

```
assert condition, "error message"
```

测试给定条件是否满足

若为 True, 则什么也不做

若为 False, 则抛出带给定错误信息的 AssertionError

抛出异常

简便方法：抛出 AssertionError

```
a = 2
assert a == 1, 'a should be 1'
print(a)
```

⊗ 0.0s Python

```
-----
AssertionError                                Traceback (most recent call last)
/var/folders/1p/9bcd1_s73b6cmjvy82yc_vr0000gn/T/ipykernel_99575/72001909.py i
n <module>
  1 a = 2
----> 2 assert a == 1, 'a should be 1'
      3 print(a)

AssertionError: a should be 1
```

```
a = 1
assert a == 1, 'a should be 1'
print(a)
```

✓ 0.0s Python

1

若为 False,

- 抛出带给定错误信息的
AssertionError

若为 True

- 什么都不做

用 assert 语句更新代码

```
def fibonacci(N):  
    if N < 0:  
        raise ValueError("N must be non-negative")  
    L = []  
    a, b = 0, 1  
    while len(L) < N:  
        a, b = b, a + b  
        L.append(a)  
    return L  
  
print(fibonacci(10))  
print(fibonacci(-10))
```

⊗ 0.2s



函数

■ 动机：从脚本到 AI 系统

回顾：

- 数据类型（整数、字符串、列表、字典等）。
- 控制流（if/else、for/while 循环、...）。
- 编写简单、线性的脚本以完成任务。

接下来：

- 编写复杂、有组织且可复用的代码。
- 构建可维护、可扩展的系统。
- 这是从“写脚本”迈向“工程化 AI 应用”的桥梁。

函数是这座桥梁的支柱。

■ 什么是函数？ 核心构建块

函数是可复用的、具名代码块，用于完成特定任务。

类比：咖啡机

- **输入（参数）**：你提供咖啡豆和水。
- **函数（机器）**：执行一套过程（研磨、加热、冲泡）。
- **输出（返回值）**：你得到一杯咖啡

inputs $\Rightarrow$ function $\Rightarrow$ outputs

函数的作用

函数围绕若干关键原则组织代码，以提高清晰度、效率与可维护性。

- **模块化（DRY 原则）：**将复杂问题分解为更小、可管理、自包含的单元。
- **可复用性：**逻辑定义一次，可在多种场景中使用，节省时间并减少重复。
- **可读性：**让代码自我说明。例如，`compute_loss(predictions, actuals)` 远比原始数学运算更直观。
- **抽象：**将复杂实现细节隐藏在简单接口之后。使用者只需知道函数*做什么*，不必知道*如何做*。

AI 示例：神经网络层就是一个函数。它封装复杂的矩阵运算，接收输入并产生输出，而不暴露内部逻辑。

函数的结构

我们来剖析 Python 中定义函数的语法。

```
1  # --- 1. Keyword  2. Name  3. Parameters -----
2  def greet(first_name, last_name):
3      """
4      4. Docstring: Explains what the function does.
5      """
6  # --- 5. Body: Indented code that performs the task ---
7      full_name = first_name + " " + last_name
8      message = "Hello, " + full_name + "!"
9  # -----
10     return message # 6. Return: Sends a value back
```

- **def**: 开始函数定义的关键字。
- **名称**: 函数的唯一标识符 (例如 **greet**) 。
- **形参**: 函数需要接收的数据的占位符。
- **Docstring**: 描述函数用途的可选字符串。
- **函数体**: 调用时执行的缩进代码块。
- **return**: 用于退出函数并可选择返回一个值的关键字

定义与调用函数

用 def 定义；通过名称加括号调用。

```
1  # --- Definition ---
2  def calculate_accuracy(predictions, labels):
3      """Calculates classification accuracy."""
4      correct = 0
5      for pred, label in zip(predictions, labels):
6          if pred == label:
7              correct += 1
8      return correct / len(labels)
9
10 # --- Calling ---
11 model_preds = ['cat', 'dog', 'cat', 'cat']
12 true_labels = ['cat', 'cat', 'cat', 'dog']
13 acc = calculate_accuracy(model_preds, true_labels)
14 print(f"Model accuracy: {acc:.2f}") # Model accuracy:
    0.50
```

■ 变量作用域：LEGB 规则

作用域决定变量可以在何处被访问。

Python 按此顺序查找变量：局部 → 嵌套（Enclosing） → 全局 → 内置（LEGB）

- **局部**：在当前函数内部定义的变量。
- **嵌套（Enclosing）**：外层函数局部作用域中的变量（用于嵌套函数）。
- **全局**：在脚本/模块顶层定义的变量。
- **内置**：Python 中预先赋值的名称（例如 print、len）

```
1 x = "global" # Global scope
2 def outer_func():
3     y = "enclosing" # Enclosing scope
4     def inner_func():
5         z = "local" # Local scope
6         print(z, y, x) # Accesses all three scopes
7     inner_func()
8 outer_func() # Prints: local enclosing global
```

■ 作用域：名称遮蔽

名称遮蔽：若局部变量与外层作用域中的变量同名，以内层变量为准

```
1 value = 10 # Global variable
2 def foo():
3     value = 3 # Local variable, shadows the global one
4     print(f"Inside foo: {value}")
5     return value
6 print(f"Before calling foo: {value}")
7 foo_result = foo()
8 print(f"After calling foo: {value}") # Global value is
   unchanged
9 print(f"Foo returned: {foo_result}")
```

Output:

```
Before calling foo: 10
Inside foo: 3
After calling foo: 10
Foo returned: 3
```

■ 修改外层作用域：global 和 nonlocal

若要修改外层作用域中的变量，必须显式声明

global 关键字：

- 修改全局作用域中的变量。
- 请谨慎使用！它会使代码难以调试。

nonlocal 关键字：

- 修改最近一层封闭作用域中的变量（但不包括 global）

```
1 TRAINING_EPOCH = 0
2 def run_training_step():
3     global TRAINING_EPOCH
4     # ... run one step ...
5     TRAINING_EPOCH += 1
```

```
1 def create_counter():
2     count = 0
3     def increment():
4         nonlocal count
5         count += 1
6         return count
7     return increment
```

return 语句

return 语句立即退出函数，并将值回传给调用者
提前退出模式（守卫子句）

```
1 def find_first_high_confidence(predictions, threshold=0.9):
2     for pred in predictions:
3         if pred['confidence'] >= threshold:
4             return pred # Exit as soon as we find one!
5     return None # Return None if loop finishes (no match)
```

返回多个值

- 可以用逗号分隔返回多个值。Python 会自动将它们打包成元组。

```
1 def get_model_stats(history):
2     loss = history['loss'][-1]
3     accuracy = history['accuracy'][-1]
4     return loss, accuracy # Returns a tuple: (loss, accuracy)
5 # Unpack the returned tuple into variables
6 final_loss, final_acc = get_model_stats(model_history)
```

文档字符串

文档字符串：函数头部之后的第一个字符串

- 可以是任意字符串（用 `'', ''', ''''', '''''', ''''''', '''''''', '''''''''` 创建）
- 通过函数的 `__doc__` 属性访问（函数是对象，具有属性）
- 供 `help()` 函数及其他工具生成文档使用
- 好的 docstring 应说明用途、参数、返回值与异常。

```

1 def preprocess_image(image, target_size=(224, 224)):
2     """Resizes and normalizes an image for model input.
3     Args:
4         image: A NumPy array representing the image.
5         target_size (tuple): The desired (width, height)
6         .
7     Returns:
8         The preprocessed image as a NumPy array.
9     """
10    # ... implementation ...
11    return processed_image

```

- 参数和返回值的可选注解。
- 运行时不强制检查；面向开发者和工具（IDE、静态分析）。
- 对大型协作 AI 项目至关重要，有助于避免缺陷。

```
1 from typing import List, Tuple, Dict, Any
2 def predict(model: Any,
3             data: List[Dict[str, float]]) -> List[str]:
4     """
5     Generates predictions for a batch of data.
6     Args:
7         model: A trained machine learning model object.
8         data: A list of dictionaries, where each dict is a
9              feature vector.
10    Returns:
11        A list of predicted labels (strings).
12    """
13    # ... implementation ...
14    return predictions
```

练习

任务：编写函数 `normalize_features`。

- 它应接收一个数字列表（例如像素值、传感器读数）。
- 它应找出列表中的最小值和最大值。
- 它应返回一个新列表，其中每个值按以下公式缩放到 0 到 1 之间：

$$\text{normalized_value} = \frac{\text{value} - \text{min_val}}{\text{max_val} - \text{min_val}}$$

- 包含 docstring 和类型提示。
- 若 `max_val == min_val` 应如何处理？请处理该边界情况

Example call:

```
1 data = [10, 20, 30, 40, 50]
2 normalized_data = normalize_features(data)
3 # expected: [0.0, 0.25, 0.5, 0.75, 1.0]
```

形参与实参

一个细微但重要的区别：

- **参数是函数定义中的命名变量，作为输入的占位符。**
- **实参是调用函数时实际传入的值**

```
1 # 'a' and 'b' are PARAMETERS
2 def add(a, b):
3     return a + b
4
5 # 5 and 10 are ARGUMENTS
6 result = add(5, 10)
```

参数传递：位置参数 vs. 关键字参数

位置参数

- 按顺序匹配到参数。
- 含义完全取决于位置

关键字参数

- 你显式指定哪个参数获得哪个值。
- 顺序不再重要。
- 使代码更易读、更不易出错。

```
1 def create_user(name, age,
2                 role):
3     # ...
4     # 'john' -> name, 30 -> age,
5     # etc.
6 create_user('john', 30, '
7             admin')
```

```
1 create_user(
2     role='admin',
3     name='john',
4     age=30
5 )
```

参数传递：位置参数 vs. 关键字参数

位置参数

- 按顺序匹配到参数。
- 含义完全取决于位置

关键字参数

- 你显式指定哪个参数获得哪个值。
- 顺序不再重要。
- 使代码更易读、更不易出错。

```
1 def create_user(name, age,
2                 role):
3     # ...
4     # 'john' -> name, 30 -> age,
5     # etc.
6 create_user('john', 30, '
7             admin')
```

```
1 create_user(
2     role='admin',
3     name='john',
4     age=30
5 )
```

规则：函数调用中，位置参数必须始终位于关键字参数之前

`create_user(name='john', 30, 'admin')` → **SyntaxError**.

默认参数

可以为参数提供默认值，使其对调用者成为可选

```
1 # AI Use Case: A flexible model training function
2 def train_model(data, epochs=10, learning_rate=0.001):
3     print(f"Training for {epochs} epochs with lr={
4         learning_rate}")
5     # ... training logic ...
6 # --- Calling the function ---
7 # 1. Use all defaults (most common case)
8 train_model(my_training_data)
9 # 2. Override 'epochs' by its position
10 train_model(my_training_data, 50)
11 # 3. Override 'learning_rate' by its keyword (more
12     readable)
13 train_model(my_training_data, learning_rate=0.01)
```

规则：函数定义中，带默认值的参数必须位于无默认值的参数之后

■ 可变默认参数的陷阱

警告：切勿将可变类型（如列表或字典）用作默认参数。这是经典的 Python 陷阱！默认对象只创建一次（在函数定义时），之后所有未提供该参数的调用都会复用它。

```
1  # WRONG WAY - The list is shared across calls
2  def add_log(message, logs=[]):
3      logs.append(message)
4      print(logs)
5  add_log("Starting process...") # >> ['Starting process...']
6  add_log("Step 1 complete.")    # >> ['Starting process...', 'Step 1
   complete.']
7  \pause
8  # CORRECT WAY - Create a new list each time if needed
9  def add_log_correct(message, logs=None):
10     if logs is None:
11         logs = []
12     logs.append(message)
13     print(logs)
14 add_log_correct("Starting process...") # >> ['Starting process...']
15 add_log_correct("Step 1 complete.")    # >> ['Step 1 complete.']
```

■ 可变长度位置参数: *args

*args 语法允许函数接受任意数量的额外位置参数，并收集到一个元组中

```
1 def process_data(required1, required2, *args):
2     print(f"Required 1: {required1}")
3     print(f"Required 2: {required2}")
4     print(f"Other args: {args}") # 'args' is a tuple
5 process_data('model_A', 'dataset_X')
6 # >> Other args: ()
7 process_data('model_B', 'dataset_Y', 'option1', 123,
8             True)
9 # >> Other args: ('option1', 123, True)
10 # AI Use Case: A function to average multiple loss
11 # values
12 def average_loss(*losses):
13     if not losses: return 0
14     return sum(losses) / len(losses)
```

■ 可变长度关键字参数: **kwargs

**kwargs 语法允许函数接受任意数量的额外关键字参数，并收集到一个字典中

```
1 def create_model_config(model_type, **kwargs):
2     config = {'type': model_type}
3     # kwargs is a dictionary of all other keyword arguments
4     config.update(kwargs)
5     return config
6 # 'layers' and 'dropout' are collected into kwargs
7 config1 = create_model_config('CNN', layers=4, dropout=0.5)
8 # >> {'type': 'CNN', 'layers': 4, 'dropout': 0.5}
9 # 'optimizer', 'lr', and 'beta' go into kwargs
10 config2 = create_model_config('Transformer', optimizer='Adam', lr
    =0.0001, beta=0.9)
11 # >> {'type': 'Transformer', 'optimizer': 'Adam', ...}
```

AI 用例：非常适合向模型层、优化器（Adam(lr=..., beta1=...)）或绘图函数传递灵活的配置参数。

用 * 和 ** 解包参数

调用函数时也可以用 * 和 ** 将可迭代对象（列表、元组）和字典解包为参数。这与用 *args/**kwargs 收集参数互为逆过程。

```
1 def plot_metrics(epoch, accuracy, loss, val_accuracy, val_loss):
2     print(f"E:{epoch} | Acc:{accuracy:.2f} | Loss:{loss:.2f} | "
3           f"Val Acc:{val_accuracy:.2f} | Val Loss:{val_loss:.2f}")
4 # Data is in a list/tuple
5 epoch_data = [5, 0.92, 0.15, 0.88, 0.21]
6 plot_metrics(*epoch_data) # Unpacks list into positional args
7 # Data is in a dictionary
8 metrics_dict = {
9     'accuracy': 0.95, 'loss': 0.11,
10    'val_accuracy': 0.91, 'val_loss': 0.18
11 }
12 # Unpacks dict into keyword args
13 plot_metrics(10, **metrics_dict)
```

■ 仅限关键字与仅限位置

Python 允许你约束参数必须如何传递，从而构建更稳健的 API。

- 仅限关键字 (* 之后)：强制按关键字传递参数。提高清晰度，尤其适用于标志参数
- 仅限位置 (/ 之前)：禁止按关键字传递参数。在参数名可能变更时很有用。

```
1 # 'verbose' and 'log_file'  
    must use keywords  
2 def process_files(*files,  
    verbose=False, log_file  
    =None):  
3     # ...  
4 process_files('a.txt',  
    verbose=True) # OK  
5 # process_files('a.txt',  
    True) # TypeError!
```

```
1 # 'x' and 'y' must be  
    positional  
2 def my_func(x, y, /):  
3     # ...  
4 my_func(1, 2) # OK  
5 # my_func(x=1, y=2) #  
    TypeError!
```

完整的参数顺序

定义函数时，参数必须按此特定顺序排列

Function Definition Syntax

```
def name(pos_only, /, standard, *args, kw_only, **kwargs):
```

- 仅位置参数：只能按位置传入。（以 / 结尾）
- 普通参数：可按位置或关键字传入。（例如 data、epochs=10）
- *args：收集所有额外的位置参数。
- 仅关键字参数：必须按关键字传入。（出现在 *args 或单独的 * 之后）
- **kwargs：收集所有额外的关键字参数。

不必全部使用，但所用参数必须按此顺序

■ 为何这个顺序很重要

直观理解：该顺序可避免实参匹配时的歧义。

- 位置参数在前，以便 Python 从左到右匹配。
- 然后是默认参数和 *args，以便额外的值有明确位置。
- 仅限关键字参数必须按名称传递，以避免混淆。
- 最后是 **kwargs，用于捕获其余所有内容。

What could go wrong without order?

```
1 # Ambiguous: is 'admin' a keyword-only arg or part of *args?
2 def create_user(*args, role, **kwargs): ...
3 create_user('john', 'admin') # Clear with enforced order
4
5 # Another ambiguity: where to assign values if order is not fixed?
6 def f(a=1, *args, b, **kwargs): ...
7 f(2, 3, b=4) # Without order, is 3 for *args or b?
```

要点：固定顺序使函数定义可预测、可读且安全。

目标：利用 Python 的参数传递特性，编写灵活的 AI 训练配置函数。

任务

- 1. 初始设置：定义 `configure_training(model_name, dataset, epochs=10)`。函数应打印其参数。仅用两个必选参数调用它。**
- 2. 优化器设置：为函数添加 `**kwargs` 以接收额外优化器参数（例如 `learning_rate=0.01`）。打印该设置字典。用自定义学习率调用它。**
- 3. 最终打磨与解包：重构使 `epochs` 成为仅限关键字参数。创建优化器设置字典，调用函数时用 `**` 将字典解包为参数。**



匿名 (Lambda) 函数

函数是一等对象

- 函数是对象
- 函数名可视作存储该对象的变量

```
def add(a, b):  
    str(1)  
    c = a + b  
    return c  
print(add)
```

✓ 0.0s

<function add at 0x10b0b5280>

函数是一等对象

- 在 Python 中，函数与其他变量（例如整数或列表）一样被对待。
- 这是一个强大的概念，称为“一等函数”。
- 这意味着你可以：
 - 将函数赋给变量。
 - **将函数作为参数传给另一个函数（高阶函数）。**
 - **从另一个函数返回一个函数。**
 - **将函数存储在列表或字典等数据结构中。**

```
1 def square(x): return x * x
2 def cube(x): return x * x * x
3 # 1. Assign to a variable & store in a dict
4 math_ops = {'sq': square, 'cb': cube}
5 print(math_ops['sq'](5)) # Output: 25
6 # 2. Pass as an argument
7 def apply_operation(op, value): return op(value)
8 print(apply_operation(cube, 10)) # Output: 1000
```

进阶概念：匿名 (Lambda) 函数

问题：有时只需一个一次性的小函数。写完整的 def 语句显得冗长。

解决方案：lambda 函数

语法： 01. `lambda parameters: return_expression`

- lambda 在单行上定义。
- 可以有多个参数，但只能有一个表达式。
- 表达式的结果会被隐式返回。

变量 (名称)

参数

`square = lambda x: x * x` 返回值

`print(square(4))`

用该变量调用函数



```
def square(x):  
    return x * x  
print(square(4))
```

要点：lambda 适合简短操作，常用于把函数作为参数传递。复杂逻辑请始终使用 def

■ Lambda 实战: map 与 filter

Lambda 与高阶函数结合使用时最为强大。

map(function, iterable): 对可迭代对象的每一项应用函数。

```
1 # AI Example: Convert raw scores (logits) to
  probabilities
2 logits = [-2.2, -1.4, 0.8, 2.0]
3 probabilities = list(map(lambda x: 1 / (1 + 2.718**(-x)),
  logits))
```

filter(function, iterable): 返回可迭代对象中使函数返回 True 的那些项

```
1 # AI Example: Filter out low-confidence predictions
2 predictions = [{'conf': 0.98}, {'conf': 0.65}, {'conf':
  0.91}]
3 high_conf = list(filter(lambda p: p['conf'] > 0.9,
  predictions))
```

■ Lambda 实战: sorted

Lambda 与高阶函数结合使用时最为强大。

sorted(iterable, key=..., reverse=False):

- 用 lambda 作为 key 来排序复杂对象。
- 设置 reverse=True 可按降序排列

```
1 # Example: sort detections by confidence (descending)
2 detections = [
3     {'label': 'person', 'confidence': 0.85, 'box': [10,10,50,50]},
4     {'label': 'car', 'confidence': 0.98, 'box': [60,20,100,80]},
5     {'label': 'person', 'confidence': 0.91, 'box': [15,12,55,52]},
6 ]
7 sorted_by_conf = sorted(detections, key=lambda d: d['confidence'],
8                          reverse=True)
```

- d 将遍历 dections 中的元素

■ Lambda 实战: reduce

Lambda 与高阶函数结合使用时最为强大。

`reduce(function, iterable[, initializer])` (来自 `functools`)

- 将序列合并为单个值 (例如求和、求积、求最小值)。
- 有内置函数时优先用 `sum` 等; 自定义折叠再用 `reduce`

```
1 from functools import reduce
2 # Example: product via reduce
3 nums = [1, 2, 3, 4]
4 prod = reduce(lambda a, b: a * b, nums, 1)
```

- The last line performs for $t \in \{0,1,2,3\}$: $a_{t+1} = a_t \times b_t$, where $a_0 = 1$, $b_t = \text{nums}[t]$.

■ Lambda 与列表推导式

对许多 map 和 filter 操作，列表推导式通常被认为更“Pythonic”、更易读

map equivalent:

```
1 # Using map and lambda
2 numbers = [1, 2, 3, 4]
3 squared_map = list(map(lambda x: x**2, numbers))
4 # Using a list comprehension (often preferred)
5 squared_comp = [x**2 for x in numbers]
```

filter equivalent:

```
1 # Using filter and lambda
2 numbers = [1, 2, 3, 4, 5, 6]
3 evens_filter = list(filter(lambda x: x % 2 == 0, numbers))
4 # Using a list comprehension
5 evens_comp = [x for x in numbers if x % 2 == 0]
```

指南：若要创建列表，推导式通常更清晰。需要传递函数本身（而非仅结果）时，使用 map 和 filter。

■ 练习

目标：在真实的 AI 小任务中使用 lambda，并与推导式/def 方案对比

任务

- Sigmoid 概率：给定 logits，用 map + lambda 对每个值计算 $\sigma(x) = 1/(1 + e^{-x})$ 。
- 过滤预测：从含 {"label": str, "conf": float} 的字典列表中，用 filter + lambda 只保留 conf >= 0.9 的项。
- 对目标检测结果排序：用 sorted 配合 lambda 作为 key，按置信度降序排列。平局时面积更大的框排在前面。
- 用列表推导式或普通 def 函数重写其中一个子任务。哪个版本更清晰，为什么？

进阶概念：闭包

当嵌套函数记住并仍能访问外层作用域中的变量时，即使外层函数已执行完毕，也会形成闭包。

```
1 def make_multiplier(n):
2     # 'n' is a "free variable" from the enclosing scope
3     def multiplier(x):
4         return x * n # multiplier remembers 'n'
5     return multiplier
6 # 'times3' is a closure that remembers n=3
7 times3 = make_multiplier(3)
8 # 'times5' is a closure that remembers n=5
9 times5 = make_multiplier(5)
10 print(times3(10)) # Output: 30
11 print(times5(10)) # Output: 50
```

AI 关联：闭包是装饰器等概念的基础，也用于在框架中创建专用、已配置的函数（例如带特定权重的损失函数）。

进阶概念：装饰器

装饰器是一个函数：它接受另一个函数作为输入，为其添加功能后返回修改后的函数，且不永久改动原函数代码。

```
1 # A simple decorator to time a function's execution
2 import time
3 def timer(func):
4     def wrapper(*args, **kwargs):
5         start_time = time.time()
6         result = func(*args, **kwargs)
7         end_time = time.time()
8         print(f"{func.__name__} took {end_time - start_time:.4f}s")
9         return result
10    return wrapper
11 @timer
12 def train_model():
13     """Simulates a time-consuming training process."""
14     time.sleep(2)
15     print("Model training complete.")
16 train_model() # This will now be timed automatically!
```

输出：
Model training complete.
train_model took 2.0011s

■ 练习

目标：实现参数化闭包和可复用装饰器，再将它们组合起来。

任务：

- 闭包：编写 `make_power(n)`，返回一个将输入 `x` 升到 `n` 次方的函数。
- 装饰器：编写 `log_call(func)`，每次调用时打印函数名、参数和返回值。
- 组合：使用装饰器为 `make_power` 生成的闭包记录调用日志。

问题：

- 为什么 `make_power` 返回后，`power` 仍能看到 `n`？
- 装饰器如何在不改变闭包逻辑的情况下添加日志？

异常

- 异常层次结构
- 异常处理：try、except、else、finally、raise、assert

函数

- 基础：定义/调用、变量作用域、return 语句、docstring、类型注解
- 形参/实参：位置参数 vs. 关键字参数、默认值、可变长度（*args、**kwargs）、仅限关键字与仅限位置（* 和 /）、参数顺序
- Lambda 函数：与高阶函数（map、filter、sorted、reduce）配合使用

Python 编程与人工智能

感谢聆听!